ملاحظات الدرس — لك

البحث

البرمجة، المستوى ٣، الدرس ٣

المدة
٢٠ دقيقة

المواد
هاتف وعليه هذا الدرس بعد تنزيله، قاموس ورقي أو سجلّ مرتّب (اختياري)

أهداف التعلّم

- تكتب البحث الخطّي.
- تشرح البحث الثنائي، ولماذا يجب أن تكون القائمة مرتّبة.
- تعدّ أسوأ حالة من النظرات لـ ١٠٠٠ عنصر.

النشاط: اعثر على البطاقة

١. التمهيد (٣ دقائق): يضع شريكك ١٦ بطاقة مرقّمة مقلوبة بترتيب عشوائي. اعثر على العدد الذي اختاره بقلب البطاقات واحدة واحدة، وعُدّ مرات القلب.
٢. المرتّبة (٧ دقائق): الآن رتّب البطاقات. اقلب البطاقة الوسطى، ثم استبعد النصف الذي لا يمكن أن يحوي العدد. وكرّر.
٣. العدّ (٧ دقائق): جرّب كل طريقة ثلاث مرات واكتب عدد مرات القلب. أيّهما لا تحتاج أبدًا إلى أكثر من ٥؟
٤. الشرح (٣ دقائق): اخلط البطاقات وجرّب التنصيف. ما الذي يخطئ؟

تصلح ورق اللعب أو مربّعات ورق مرقّمة. ولا حاجة إلى هاتف حتى مرحلة الكود.

للنقاش

- لماذا يحتاج البحث الثنائي إلى قائمة مرتّبة؟
- متى يكفي البحث الخطّي؟
- لماذا تحتاج قائمة أكبر بألف مرة إلى نحو ١٠ نظرات إضافية فقط؟

تحقّق من نفسك

- كم نظرة لـ ١٠٠٠ عنصر: أسوأ حالة للخطّي، وأسوأ حالة للثنائي؟

جرّبها هذا الأسبوع

العب «خمّن رقمي» من ١ إلى ١٠٠٠ مع صديق بالتنصيف. هل تستطيع دائمًا أن تجده في ١٠ تخمينات؟

ورقة عمل

البحث

البرمجة، المستوى ٣، الدرس ٣

الاسم _____ الفصل _____ التاريخ _____

١ **قائمة مرتّبة فيها ١٠٢٤ اسمًا. كم نظرة يحتاج إليها البحث الثنائي على الأكثر؟**

○ ١٠٢٤ ○ ١١ ○ ٥١٢

٢ **اكتب find(names, target): بحث خطّي. تعيد موضع أول تطابق، أو -1 إن لم يكن الهدف موجودًا.**

```
def find(names, target):
    # position of target, or -1
    return -1
```

اكتب برنامجك:

٣ **رتّب البحث الثنائي. يحدّد lo وhig الجزء الذي ما زال ممكنًا من القائمة المرتّبة. وفيها سطر لا مكان له.**

رقّم الأسطر بالترتيب (١، ٢، ٣...)، وارسم أمام السطر سهمًا → عن كل إزاحة يحتاج إليها. اشطب أي سطر لا ينتمي إلى البرنامج.

☐ `mid = (lo + hi) // 2`

☐ `if a[mid] == x:`

☐ `while lo < hi:`

☐ `return mid`

☐ `elif a[mid] < x:`

☐ `def search(a, x):`

☐ `hi = mid - 1`

☐ `else:`

☐ `lo, hi = 0, len(a) - 1`

☐ `lo = mid + 1`

☐ `while lo <= hi:`

☐ `return -1`

٤ **ماذا يطبع هذا البرنامج؟**

```
names = ["Ada", "Kofi", "Layla", "Omar"]
print(names.index("Omar"))
```

○ 3 ○ Omar ○ 4

٥ **أيّها صحيح عن البحث الثنائي؟**

☐ كل نظرة تنصّف ما بقي ☐ يعمل على أيّ قائمة، مرتّبة أو غير مرتّبة ☐ يحتاج ١٠٠٠ عنصر إلى نحو ١٠ نظرات على الأكثر

☐ يجب أن تكون القائمة مرتّبة

✎ **أين تبحث في حياتك في قائمة مرتّبة (قاموس، أو جهات اتصال، أو سجلّ)؟ وكيف تبحث فيها: واحدًا واحدًا، أم بالتنصيف؟**

تحقّق من إجاباتك

البحث

١ **١١**

تنصيف ١٠٢٤ يصل إلى ١ بعد ١٠ تنصيفات، ثم نظرة أخيرة إلى الاسم الأخير: ١١.

٢

```
def find(names, target):
    for i in range(len(names)):
        if names[i] == target:
            return i
    return -1
```

أعِد الموضع لحظة إيجاد الهدف، ولا تُعِد -1 إلّا بعد أن تفحص الحلقة كل عنصر.

٣ **من الأعلى إلى الأسفل: ٤، ٥، ✗، ٦، ٧، ١، ١٠، ٩، ٢، ٨، ٣، ١١**

```
def search(a, x):
    lo, hi = 0, len(a) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if a[mid] == x:
            return mid
        elif a[mid] < x:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1
```

انظر إلى الوسط، فإن كان الهدف أكبر فانقل lo إلى ما بعد mid، وإلّا فانقل hi إلى ما قبله.

٤ **3**

index() بحث خطّي: يفحص من البداية ويعيد الموضع، بالعدّ من 0.

٥ **يجب أن تكون القائمة مرتّبة، كل نظرة تنصّف ما بقي، يحتاج ١٠٠٠ عنصر إلى نحو ١٠ نظرات على الأكثر**

ينصّف البحث الثنائي قائمة مرتّبة مع كل نظرة.